



МОСКОВСКИЙ УНИВЕРСИТЕТ ИМЕНИ С.Ю. ВИТТЕ

Факультет Информационных технологий

Кафедра Кафедра информационных систем

Направление подготовки/Специальность Прикладная информатика

ОТЧЕТ

о прохождении

Производственная практика

/вид практики/

Эксплуатационная практика

/тип практики/

Студента Авасева Даниила Сергеевича
(фамилия, имя, отчество)

Место прохождения практики ЧОУВО «МУ им. С.Ю. Витте»

Период прохождения практики с 20.04.2026 г. по 31.05.2026 г.

г. Москва 2026 г.

СОДЕРЖАНИЕ

Введение	3
1. Основная часть	4
2. Анализ предметной области	4
2.1. Выбор и настройка среды разработки Python	4
2.2. Анализ обработки научных pdf документов	4
2.3. Анализ методов поиска математических функций и свойств	6
2.4. Использование нейросетевых моделей при обработке текста	6
2.5. Вывод по разделу	7
3. Разработка системы анализа научных текстов	8
3.1. Разработка структуры программы	8
3.2. Чтение и обработка pdf документов	9
3.2.1. Получение и чтение pdf файлов	9
3.2.2. Очистка и нормализация текста	10
3.2.3. Формирование текстовых страниц и абзацев	11
3.3. Реализация поиска ключевых слов	12
3.4. Использование нейросетевой модели через Ollama	13
3.4.1. Определение названия статьи и автора	14
3.5. Формирование итогового датасета	15
3.6. Анализ результатов работы программы	16
3.7. Вывод по разделу	17
4. Заключение	18
5. Список использованной литературы	19

Введение

Производственная практика проходила в Московском университете им. С.Ю. Витте на кафедре информационных систем и технологий. В ходе практики рассматривались вопросы обработки научных текстов и применения современных программных средств для анализа данных.

Во время выполнения работы изучались возможности языка программирования Python, библиотеки для обработки pdf документов и инструменты для работы с текстовой информацией. Отдельное внимание уделялось автоматизации поиска нужных фрагментов текста и использованию нейросетевых моделей при обработке научных статей.

Целью практики являлось получение практических навыков разработки программных решений для анализа текстовых данных и работы с современными средствами обработки информации.

В процессе практики были закреплены навыки программирования, работы с файлами, обработки текста и формирования структурированных данных для дальнейшего анализа.

Цель практики - получение знаний о возможности использования информационных технологий для решения прикладных задач, а также выработка практических навыков по их анализу, выбору и применению информационных технологий в Университете.

В ходе прохождения практики были поставлены следующие задачи:

- закрепление приобретенных теоретических знаний
- приобретение опыта разработки программных решений на языке Python;
- анализ предметной области и подготовка требований к разрабатываемой системе
- изучение методов обработки научных текстов
- сбор и обработка данных для дальнейшего анализа
- подготовка отчетной документации по результатам практики.

В ходе практики предполагается создание программного решения для автоматизированного анализа научных статей и формирования датасета,

содержащего сведения о свойствах математических функций. Основное внимание уделяется поиску фрагментов, связанных с непрерывностью, гладкостью, дифференцируемостью и разрывами функций.

Оценка результата работы будет выполняться по качеству подготовленного набора данных. В датасете должны присутствовать только содержательные текстовые фрагменты, относящиеся к исследуемым свойствам функций. Для каждого элемента необходимо сохранить название источника и сведения об авторах публикации. Отдельное внимание уделяется точности выделения фрагментов, сохранению их смыслового содержания и правильности определения границ текста, задаваемых начальными и конечными индексами.

Ссылка на github: <https://github.com/MelonWaveTeam/AvasevFunctions>

1. Основная часть

2. Анализ предметной области

2.1. Выбор и настройка среды разработки Python

Для разработки программы был выбран язык Python, так как он часто используется при обработке текстовых данных и работе с pdf документами. Также Python позволяет быстро подключать сторонние библиотеки и упрощает работу с файлами и структурами данных.

Разработка выполнялась в среде PyCharm. Данная среда была выбрана из за удобной работы с Python проектами, встроенного терминала и простой настройки библиотек. Для работы программы дополнительно были установлены библиотеки pymupdf, ollama и fitz.

В процессе настройки была подготовлена рабочая папка проекта, содержащая pdf документы, основной файл программы и файл words.jsonl, в который сохраняются результаты обработки. После установки библиотек была проведена проверка чтения pdf файлов и корректности запуска программы.

2.2. Анализ обработки научных pdf документов

Научные pdf документы достаточно сложные по структуре. В них могут находиться формулы, специальные символы, переносы строк и различные элементы оформления. из за этого при извлечении текста часто появляются лишние пробелы и ошибки в расположении предложений.

В данной работе обработка документов выполнялась при помощи библиотеки fitz. Программа открывает pdf файл, разбивает его на страницы и получает текстовые блоки для дальнейшего анализа. После этого текст очищается от переносов строк, табуляции и повторяющихся пробелов.

Особенностью обработки научных статей является то, что нужная информация может находиться в разных частях документа.

Проект включает папку dataset, в которой находятся pdf документы, основной файл программы main.py, файл README.md с описанием работы программы и файл words.jsonl для сохранения результатов обработки.

На рисунке 1. показана структура проекта и расположение основных файлов программы.

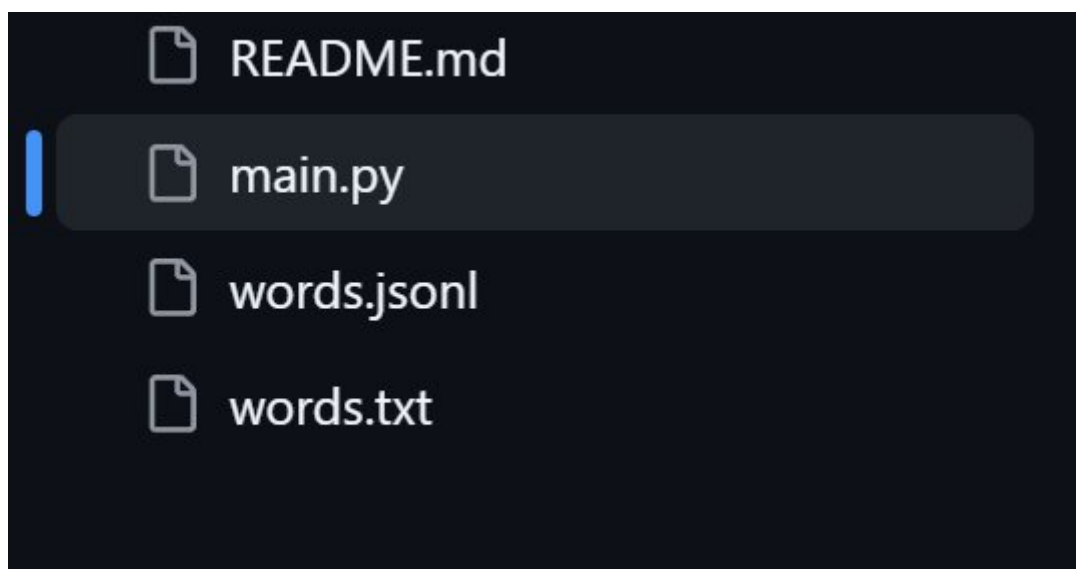


Рисунок 1.

2.3. Анализ методов поиска математических функций и свойств

При обработке научных статей одной из основных задач является поиск предложений. В текстах часто встречаются слова, связанные с непрерывностью, гладкостью, разрывами и дифференцируемостью функций.

Для поиска таких фрагментов можно использовать нейросети или поиск по ключевым словам. Полный анализ текста нейросетью требует больше времени и вычислительных ресурсов, поэтому в данной работе был выбран более простой способ поиска.

В программе используется проверка предложений по ключевым корням слов. Для этого применяются слова непрерывн, гладк, разрыв и дифференцируем. Такой подход позволяет находить предложения, связанные со свойствами математических функций, и сохранять их для дальнейшего анализа.

2.4. Использование нейросетевых моделей при обработке текста

В настоящее время нейросетевые модели активно используются для обработки текстовой информации. Они позволяют автоматически анализировать документы, находить нужные данные и извлекать информацию из больших текстовых файлов.

В данной работе нейросетевая модель применялась для определения названия научной статьи и автора по тексту первой страницы pdf документа. Для этого использовалась система Ollama и модель llama3.1:8b.

Использование нейросети позволило автоматизировать часть обработки документов и сократить количество ручной работы при формировании датасета. После получения ответа программа сохраняет найденные данные вместе с найденными предложениями в итоговый файл words.jsonl.

Такой подход упростил обработку большого количества научных статей и сделал структуру итогового датасета более полной и удобной для дальнейшего анализа.

2.5. Вывод по разделу

В ходе анализа предметной области были рассмотрены особенности обработки научных pdf документов, способы поиска математических функций и применение нейросетевых моделей при анализе текста. Также была выполнена настройка среды разработки Python и изучены библиотеки, необходимые для работы программы.

Проведённый анализ позволил определить структуру будущей программы, способы обработки текстовых данных и методы формирования итогового датасета.

Выводы	Сформированные компетенции
Проведен выбор и настройка среды разработки для разработки программы обработки научных pdf документов на языке Python	ПК-1. Способность разработки прикладного программного обеспечения, автоматизации работы с базами данных и документами, программирования бизнес-логики приложений, интеграции разнородных данных
Проведен анализ методов обработки научных текстов и поиска математических функций и свойств	
Изучены возможности применения нейросетевых моделей при анализе текстовых данных	ПК-7. Способностью использовать отечественные и международные стандарты при проектировании и обеспечении качества прикладного программного обеспечения

3. Разработка системы анализа научных текстов

3.1. Разработка структуры программы

Разработка программы началась с создания класса PdfToTxt, в котором были собраны основные этапы обработки научных pdf документов. Такой подход позволил объединить чтение файлов, обработку текста, поиск предложений и сохранение результатов внутри одной структуры.

В конструкторе класса создаются основные переменные, которые используются в дальнейшей работе программы. В них сохраняются имя файла, текст документа, список страниц, найденные результаты, название статьи и автор.

```
class PdfToTxt:
    def __init__(self, file_name):
        self.file_name = file_name
        self.all_text = ""
        self.pages_text = []
        self.results = []
        self.article_title = ""
        self.author = ""
```

После создания объекта класса запуск обработки выполняется через метод run(). Внутри него последовательно вызываются чтение pdf файла, поиск ключевых слов и сохранение результата.

```
def run(self):
    self.pdf_reader()

    if not self.all_text.strip():
        self.file_writer()
        return
    self.find_by_keywords()

    if not self.results:
        print("нет функций в файле")

    self.file_writer()
```

Такая структура упростила дальнейшую разработку программы и позволила разделить обработку документа на отдельные этапы.

3.2. Чтение и обработка pdf документов

3.2.1. Получение и чтение pdf файлов

Получение текста из PDF-файла выполняется в методе `pdf_reader()`. На этом этапе программа открывает документ через библиотеку `fitz`, получает текст первой страницы для определения названия статьи и автора, а затем переходит к обработке остальных страниц.

```
def pdf_reader(self):  
    doc = fitz.open(self.file_name)  
    global_pos = 0  
    first_page_text = doc[0].get_text()  
    self.get_metadata_from_llm(first_page_text)
```

Основной текст страницы теперь извлекается через метод `page.get_text()`. После этого он дополнительно очищается от лишних символов, которые могут появляться при чтении PDF-документов.

```
page_text_raw = page.get_text()  
page_text_raw = re.sub(r'[\u201c-\u201d]', "", page_text_raw)
```

Такой способ был выбран потому, что при работе с научными статьями важно получить не только отдельные блоки текста, но и сохранить более цельную структуру страницы для дальнейшего разбиения на абзацы.

3.2.3. Формирование текстовых страниц и абзацев

После очистки программа формирует абзацы из строк страницы. Если после прохода по строкам остался незаписанный фрагмент, он также добавляется в список абзацев.

```
if current_para:  
    paragraphs.append(''.join(current_para))
```

Затем все абзацы страницы объединяются через двойной перенос строки. Это нужно для того, чтобы дальше программа могла отделять один смысловой фрагмент от другого.

```
page_text = '\n\n'.join(paragraphs)
```

После этого данные страницы сохраняются в список `pages_text`. Для каждой страницы записываются номер, текст, начальная и конечная позиция.

```
if page_text.strip():  
    self.pages_text.append({  
        "page_num": i + 1,  
        "text": page_text.strip(),  
        "start": global_pos,  
        "end": global_pos + len(page_text.strip())  
    })
```

Такая структура позволяет дальше искать нужные абзацы и сохранять для них начальный и конечный индексы.

3.3. Реализация поиска ключевых слов

Поиск нужных фрагментов выполняется в методе `find_by_keywords()`. В новой версии программы поиск идёт не по отдельным предложениям, а по абзацам. Это удобнее для научных текстов, потому что важная мысль часто занимает несколько строк.

```
def find_by_keywords(self):  
    keywords = ['непрерывн', 'гладк', 'разрыв', 'дифференцируем']
```

Сначала программа получает текст страницы и делит его на абзацы по двойному переносу строки.

```
paragraphs = page_text.split("\n\n")  
current_pos = page_start
```

Каждый абзац проверяется на наличие ключевых слов. Если найдено совпадение, фрагмент сохраняется в список результатов.

```
for kw in keywords:  
    if kw.lower() in para.lower():  
        found = True  
        break
```

В итоговый результат записывается сам абзац, номер страницы, начальный и конечный индекс, а также краткая метка.

```
if found:  
    self.results.append({  
        "text": para,  
        "begin_index": current_pos,  
        "end_index": current_pos + len(para),  
        "page_number": page_num,  
        "summary": "функция"  
    })
```

Такой вариант поиска лучше сохраняет смысл текста, потому что в датасет попадает не обрывок предложения, а более полный фрагмент, связанный со свойствами функций.

3.4. Использование нейросетевой модели через Ollama

Для получения данных о статье в программе используется нейросетевая модель через Ollama. Она нужна не для поиска ключевых слов, а для отдельной задачи, определения названия статьи и автора по первой странице pdf файла.

Запрос к модели выполняется в методе `get_metadata_from_llm()`. Сначала формируется короткий промпт, где программе указывается вернуть результат в формате JSONL.

```
prompt = f"""извлеки из текста название статьи и автора. верни json:
{{"title": "", "author": ""}}
текст: {first_page_text[:2000]}"""
```

После этого запрос отправляется в модель `llama3.1:8b`.

```
response = chat(model='llama3.1:8b', messages=[
    {'role': 'user', 'content': prompt}
])
```

Ответ модели сохраняется в переменную `answer`, затем из него извлекается JSON-часть. Это сделано для того, чтобы программа могла получить только нужные поля, название и автора.

```
answer = response.message.content
import re
json_match = re.search(r'\{.*\}', answer, re.DOTALL)
```

Если данные удалось получить, они сохраняются в переменные класса `article_title` и `author`. В дальнейшем эти значения записываются в итоговый файл вместе с найденными предложениями.

3.4.1. Определение названия статьи и автора

Для каждой статьи программа пытается получить название и автора автоматически. Для этого берётся текст первой страницы, так как именно там находятся основные сведения о публикации.

В методе `get_metadata_from_llm()` сначала создаётся запрос к модели. В нём указано, что ответ нужно вернуть в виде JSONL.

```
prompt = f"""извлеки из текста название статьи и автора. верни json:
{{"title": "", "author": ""}}
текст: {first_page_text[:2000]}"""
```

После получения ответа программа ищет внутри него структуру и записывает его.

```
json_match = re.search(r'\{.*\}', answer, re.DOTALL)
if json_match:
    data = json.loads(json_match.group())
    self.article_title = data.get("title", "")
    self.author = data.get("author", "")
```

Если модель не смогла вернуть данные или во время обработки возникла ошибка, поля остаются пустыми. Это сделано для того, чтобы программа не останавливалась из-за одного неудачно обработанного документа.

```
except:
    self.article_title = ""
    self.author = ""
```

После этого название статьи и автор сохраняются вместе с найденными предложениями в итоговый файл `words.jsonl`.

3.5. Формирование итогового датасета

После поиска подходящих предложений программа сохраняет результат в файл `words.jsonl`. В него записываются имя обработанного pdf файла, название статьи, автор и найденные фрагменты текста.

Запись данных выполняется в методе `file_writer()`.

```
def file_writer(self):
    with open("words.jsonl", "a", encoding="utf-8") as f:
        data = {
            "file_name": self.file_name,
            "article_title": self.article_title,
            "author": self.author,
            "text": self.results
        }
        f.write(json.dumps(data, ensure_ascii=False, indent=4) + "\n")
```

Формат JSONL был выбран потому, что в него удобно добавлять результаты по каждому обработанному файлу. Каждая новая запись содержит данные по отдельной статье.

Внутри поля `text` хранится список найденных предложений. Для каждого предложения сохраняется сам текст, начальный и конечный индекс, номер страницы и краткая метка функция.

3.6. Анализ результатов работы программы

После запуска программы был сформирован файл `words.jsonl`, содержащий найденные предложения из научных pdf документов. В итоговый датасет сохранялись название статьи, автор и предложения, в которых встречались ключевые слова, связанные со свойствами математических функций.

В процессе проверки было установлено, что программа корректно обрабатывает pdf файлы, разбивает текст на страницы и находит предложения по заданным ключевым словам. Также программа успешно сохраняет номер страницы и позиции найденного текста внутри документа.

Пример результата работы программы включает поля с названием статьи, автором и списком найденных предложений. Для каждого предложения дополнительно сохраняются индексы начала и конца текста.

При анализе результатов было замечено, что качество обработки зависит от структуры pdf документа. В некоторых файлах встречались проблемы с переносами строк, символами и разделением предложений. Несмотря на это, программа смогла сформировать итоговый датасет и сохранить найденные фрагменты в удобном формате.

Пример содержимого файла `words.jsonl` после обработки научных статей.

```
{
  "file_name": "dataset/elibrary_35217063_60535409.pdf",
  "article_title": "",
  "author": "",
  "text": [
    {
      "text": "Тогда для ее решения можно использовать произвольную непрерывную функцию  $f$ , монотонно переводящую другое связное множество  $X \subset \mathbb{R}$  в  $Y$  .",
      "begin_index": 1634,
      "end_index": 1769,
      "page_number": 2,
      "summary": "функция"
    }
  ]
}
```


3.7. Вывод по разделу

В ходе разработки была создана программа для обработки научных pdf документов и формирования датасета с найденными предложениями. Программа выполняет чтение файлов, очистку текста, поиск ключевых слов и сохранение результатов в формате JSONL.

Дополнительно была реализована работа с нейросетевой моделью через Ollama для автоматического определения названия статьи и автора. Полученный датасет может использоваться для дальнейшего анализа научных текстов и обработки математических данных.

Выводы	Сформированные компетенции
Разработана программа обработки научных pdf документов на языке Python. Реализованы чтение pdf файлов, очистка текста и поиск предложений по ключевым словам Сформирован итоговый датасет в формате JSONL с найденными фрагментами текста Реализовано использование нейросетевой модели через Ollama для определения названия статьи и автора	ПК-1. Способность разработки прикладного программного обеспечения, автоматизации работы с базами данных и документами, программирования бизнес-логики приложений, интеграции разнородных данных ПК-8. Способность выполнять интеллектуальный анализ больших данных

4. Заключение

В ходе прохождения производственной практики была разработана программа для обработки научных pdf документов и формирования датасета с найденными текстовыми фрагментами. В процессе работы были изучены особенности обработки научных статей, методы анализа текстовых данных и применение языка программирования Python при решении прикладных задач.

Во время практики были получены навыки работы с pdf документами, библиотеками для обработки текста и средствами автоматизации анализа данных. Дополнительно была изучена работа с нейросетевыми моделями через Ollama для автоматического определения названия статьи и автора.

В ходе выполнения работы были реализованы чтение pdf файлов, очистка и нормализация текста, поиск предложений по ключевым словам и сохранение результатов в формате JSONL. Полученный датасет может использоваться для дальнейшего анализа научных текстов и обработки математических данных.

В процессе практики были закреплены навыки разработки программных решений на языке Python, обработки текстовой информации и формирования структурированных данных. Также были получены навыки самостоятельной работы, анализа результатов и подготовки отчетной документации.

Таким образом, цель производственной практики была достигнута, а поставленные задачи выполнены в полном объеме.

5. Список использованной литературы

1. Обработка PDF-документов в Python // Хабр. – URL: <https://habr.com/ru/amp/publications/577776/> (дата обращения: 12.05.2026).
2. Ollama: официальный сайт // Ollama. – URL: <https://ollama.com> (дата обращения: 13.05.2026).
3. Практическое руководство по работе с Ollama // Хабр. – URL: <https://habr.com/ru/articles/960256/> (дата обращения: 15.05.2026).
4. Ollama от А до Я: как выбрать модель, настроить и использовать // Хабр. – URL: <https://habr.com/ru/articles/949676/> (дата обращения: 17.05.2026).
5. Chat API Documentation // Ollama Docs. – URL: <https://docs.ollama.com/api/chat> (дата обращения: 18.05.2026).
6. Работа с PDF-файлами в Python // Хабр. – URL: <https://habr.com/ru/articles/349860/> (дата обращения: 20.05.2026).
7. PyMuPDF Documentation // PyMuPDF Documentation. – URL: <https://pymupdf.readthedocs.io/en/latest/> (дата обращения: 21.05.2026).
8. Работа с PDF-файлами в Python // Хабр. – URL: <https://habr.com/ru/articles/349860/> (дата обращения: 22.05.2026).
9. Извлечение текста с помощью PyMuPDF // PyMuPDF Documentation. – URL: <https://pymupdf.readthedocs.io/en/latest/recipes-text.html> (дата обращения: 23.05.2026).
10. Introduction API Documentation // Ollama Docs. – URL: <https://docs.ollama.com/api/introduction> (дата обращения: 24.05.2026).
11. Chat API Documentation // Ollama Docs. – URL: <https://docs.ollama.com/api/chat> (дата обращения: 25.05.2026).
12. Библиотека моделей Ollama // Ollama. – URL: <https://ollama.com/library> (дата обращения: 26.05.2026).

13. Обработка PDF-документов в Python // Хабр. – URL:
<https://habr.com/ru/amp/publications/577776/> (дата обращения:
27.05.2026).